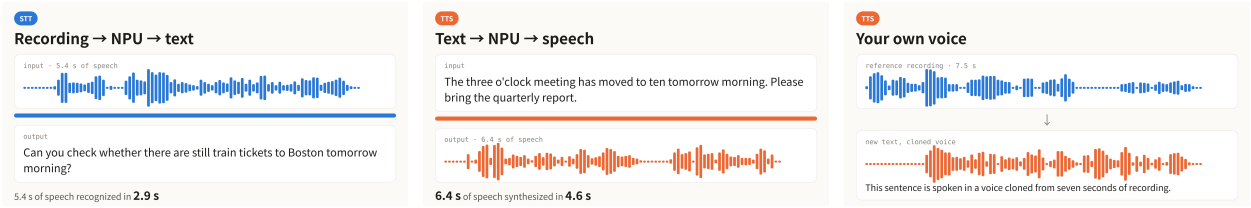
Let the Idle NPU Speak:
Local Voice Chat on a Ryzen AI Laptop

Running CosyVoice3 text-to-speech next to a speech recognizer and an LLM

Yuhua Chen
https://www.yuhuachen.com

October 2026



(a) Listening: 5.4 s of speech becomes text in 2.9 s.

(b) Speaking: 6.4 s of speech is synthesized in 4.6 s.

(c) Any voice: a 7.5 s recording becomes a new voice.

Figure 1: What npu-cosyvoice does, from the demo video: speech recognition and synthesis on the NPU of a Ryzen AI laptop, with voice cloning. Times are measured on a Ryzen 7 8845HS; the synthesis figure is the first sentence of a reply, which uses 4 flow steps (§3.4).

Abstract

Local voice chat with a large language model needs speech recognition and speech synthesis next to the model itself, and on a laptop all three compete for the same CPU cores and memory bandwidth. AMD Ryzen AI laptops carry a neural processing unit (XDNA1) that is idle in most deployments. We move the most expensive part of CosyVoice3 text-to-speech, its 22-block flow-matching DiT decoder, onto that NPU. Every projection GEMM of the DiT blocks runs as an int8×int8→int32 multiplication through one fixed 512×1024×1024 MLIR-AIE kernel, with W8A8 quantization and SmoothQuant on the query/key/value input; normalization, rotary embedding, attention and activations stay in AVX-512 C++ and overlap with the NPU. We describe how one kernel covers the whole model, why a first offload was slower than the CPU, how the language model is moved to llama.cpp without changing CosyVoice's own token handling, and how synthesis yields the NPU to a speech recognizer (whisper-xdna) at layer boundaries. On a Ryzen 7 8845HS a DiT block takes 7.4 ms (6 host threads) against 20.0 ms on the CPU in bf16 (8 threads); at the full 10 flow steps the mel spectrogram deviates 3.0% from the bf16 reference, where a CPU deployment that halves the steps to keep up deviates 7.4%. Under identical conditions the real-time factor of sentence synthesis falls from 1.44 to 0.84, and overlapping the language model with the flow lowers it further (0.85 to 0.64 in a separate configuration); speech recognition next to active synthesis is within about 3% of recognition alone. Code, a docker-compose bundle exposing both services through the OpenAI audio API, and audio samples are open source under Apache-2.0.

1 Introduction

Voice is the most natural interface to a chat model, and running it locally keeps audio on the device. A local voice loop has three models: speech-to-text (STT), the chat LLM, and text-to-speech (TTS). On a laptop-class system-on-chip they share one memory bus, and the LLM, which dominates perceived latency, is memory-bandwidth bound. Every core and every byte per second that STT and TTS take is taken from the LLM. Large models now fit on such machines: JustFit [3] serves a 27B model over its full 256K-token context in 24 GiB on a laptop, by managing model state just in time. Once the LLM fits, the question becomes what a voice costs next to it.

Modern TTS systems are not small. CosyVoice3 [7], a strong open multilingual model with zero-shot voice cloning, generates speech tokens with a 0.5B-parameter Qwen2 language model, converts them into a mel spectrogram with a flow-matching [12] diffusion transformer (DiT [14]) run with classifier-free guidance [9] for 10 Euler steps, and renders audio with a HiFT vocoder. On an 8-core Zen 4 CPU the flow decoder alone takes about 4 s per sentence at 10 steps, slower than real time, and CPU deployments commonly cut it to 5 steps at a measurable cost in quality.

Ryzen AI 7040/8040 processors include an XDNA1 NPU, a 4-column array of AI Engine tiles rated at 16 TOPS, with a mainline Linux driver [1] since 6.14. It sits idle in most Linux setups. This report describes how we run CosyVoice3's flow decoder on it, together with the decisions that made the result usable in a real voice loop:

- **One kernel for every projection.** The six projection GEMMs of each DiT block are decomposed onto a single compiled $512\times1024\times1024$ int8 kernel on one NPU hardware context, and the host work is overlapped with it; attention and the other operations stay on the CPU (§3.1, §3.2).
- **W8A8 with targeted SmoothQuant.** A sensitivity study finds the query/key/value projection responsible for most of the int8 error; SmoothQuant [16] on that one input brings the error from 3.4% to 2.3% in simulation (§3.3).
- **Fitting next to other NPU and CPU users.** Synthesis cooperatively yields the NPU to a speech recognizer at layer boundaries; the LM runs through llama.cpp [8] with separate prefill and decode thread counts; flow steps adapt to the playback deadline (§3.4).

Section 4 evaluates latency, quality and coexistence with STT and a 27B LLM on the same chip, and Section 5 lists what we have not shown.

2 Background

CosyVoice3 inference. For each sentence, the LM autoregressively produces speech tokens at 25 tokens per second of audio. The flow decoder maps them, together with the reference voice's prompt features, to an 80-bin mel spectrogram at 50 frames per second by integrating a learned velocity field: the estimator, a DiT with 22 blocks of width 1024 (16 heads of 64, feed-forward width 2048, AdaLayerNormZero modulation [14]), is evaluated 10 times, each on a batch of 2 for classifier-free guidance. Zero-shot voice cloning conditions the LM and flow on a few seconds of reference audio and its transcript; the reference prompt is part of the flow's input sequence (in a typical sentence, 174 of 478 frames).

XDNA1. The NPU is programmed by loading an *overlay* (an xclbin plus an instruction stream) into a *hardware context*. We measured a ceiling of 6 concurrent hardware contexts per device: the seventh `CREATE_HWCTX` fails with `EINVAL`. Switching the overlay that a context runs costs 1.3–1.4 ms even for the same xclbin, and there is no preemption between contexts. We generate the GEMM overlay with MLIR-AIE's whole-array matrix-multiplication design [2] and drive it through XRT.

Table 1: Where each part of CosyVoice3 runs.

Part	Runs on
LM backbone (Qwen2, 0.5B)	CPU, llama.cpp Q4_0; 4 threads prefill, 2 decode
LM embeddings, speech-token head, sampling	CPU, PyTorch (unchanged)
DiT blocks: q/k/v, output, 2 feed-forward GEMMs	**NPU**, int8×int8→int32, one overlay
DiT blocks: LayerNorm+AdaLN, RoPE, attention, GELU, (de)quantization	CPU, AVX-512 C++, 4–6 threads
time/input embedding, output projection, HiFT vocoder	CPU, PyTorch (unchanged)

Table 2: One DiT block (CFG batch 2 × 478 frames), host threads 4 / 6.

Version	LLM idle (ms)	27B LLM generating (ms)
CPU bf16, PyTorch, 8 threads (reference)	20.0	–
NPU int8 GEMMs + naive fp32 attention	26.3 / 20.4	–
+ AVX-512 attention	11.0 / 9.6	20.2 / 18.0
+ bf16 attention dot products	9.2 / 8.2	15.5 / 14.0
+ fused stages, cached AdaLN modulation	**8.3 / 7.4**	**12.4 / 11.2**

3 Design

3.1 Partitioning the DiT

Table 1 shows where each part of the pipeline runs. Only the flow estimator's transformer blocks move to the NPU; the time and input embeddings, output projection and vocoder stay in PyTorch, since they are a small share of the time.

Why a first offload was slower. A first prototype that ran bf16 GEMMs on the NPU from Python was *slower* than the CPU (31.5 ms against 20.0 ms per block). The GEMMs themselves took 13.9 ms (about 1.15 TFLOPS); the rest was about 4 ms of transfers and 11 ms of host operations, during which the NPU sat idle. Offloading only pays if the GEMMs get cheaper *and* the host work shrinks enough to hide behind them. int8 addresses the first: the same GEMMs take 4.6 ms per block in int8 against 10.7 ms in bf16 (2.3×), widening to 2.7× when an LLM is generating on the iGPU, since int8 moves half the bytes. The host-side paragraph below and Section 3.2 address the second.

Host side. The remaining operations run in one C++ library with a spinning thread pool: per-row LayerNorm with AdaLN modulation fused with activation quantization, RoPE and bf16 packing fused with dequantization, attention with AVX-512 bf16 dot products and a vectorized softmax, and GELU fused with requantization. The block is split into five host stages, each ending in NPU launches, and the two classifier-free-guidance halves are interleaved stage by stage (Figure 2): while the NPU multiplies one half, the host prepares the other, and it only waits when it needs a result that is not ready. Table 2 traces how shrinking the host stages brought the block from 26.3 ms to 7.4 ms. At the kernel's standalone rate, the 16 launches of a 478-frame block amount to about 6.4 ms, so at 7.4 ms the NPU is busy most of the time.

3.2 One kernel for the whole DiT

Each overlay switch costs 1.3–1.4 ms, and a block has only a few milliseconds of GEMM work, so the DiT must run on one overlay without switching. We compile a single kernel, $C_{512\times1024} = A_{512\times1024}\, B_{1024\times1024}$

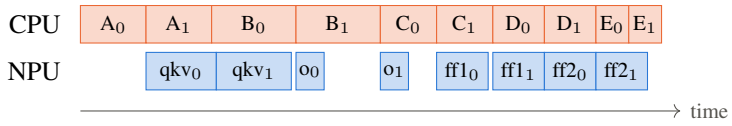


Figure 2: Schematic of one DiT block (not to scale). Host stages: A = LayerNorm, AdaLN, smoothing and quantization; B = dequantization, RoPE, attention, quantization; C = output residual, second norm, quantization; D = GELU and requantization; E = final residual. Subscripts are the two classifier-free-guidance halves. Each host stage ends by launching the NPU work of that half, and the host continues with the other half instead of waiting. Before the host-side optimizations, the CPU stages were several times longer than the NPU bars and the NPU spent most of the block idle.

in int8 with int32 output (64×64×64 tiles over the whole array; 0.40 ms, 2.7 effective TOPS, bit-exact against numpy), and map every projection onto it (Table 3):

Table 3: Every projection of a DiT block expressed as launches of one 512×1024×1024 kernel, per 512-row chunk and CFG half.

Projection	$K \times N$	Launches	How
q, k, v	1024×3072	3	one quantized activation shared by three weight tiles
attention output	1024×1024	1	direct
feed-forward 1	1024×2048	2	split along N; outputs side by side
feed-forward 2	2048×1024	2	split along K; int32 partial sums added on the host

- **Splitting** N (feed-forward 1) is free: two launches write two output buffers that the host reads as one row.
- **Splitting** K (feed-forward 2) needs a reduction. Both halves use the same per-row activation scale, so the two int32 partial products are added exactly before dequantization; no precision is lost.
- **Sharing the activation** (q/k/v) means one quantization per row feeds three launches. This is also why the deployed SmoothQuant uses one smoothing factor for q, k and v (§3.3).
- **Rows** are processed in 512-row chunks; a shorter last chunk leaves stale rows in the buffer whose outputs are simply not read. Sequences up to 1536 frames (3 chunks) need no recompilation.
- **Buffers.** The 176 weight tiles of the 22 blocks (178 MB of int8) are uploaded once as device buffers. Per CFG half and chunk, two activation buffers, three output buffers and the kernel-run objects are created once and reused on every call, so a forward pass allocates nothing. Dequantization multiplies by the row's activation scale and the column's weight scale and adds the bias, fused into the next host stage.

A 478-frame block therefore costs $2 \times 8 = 16$ launches, 352 per estimator call.

Errors never abort. Driver errors are returned to Python, which retries the step; a forward pass writes its output only on success. This mattered in practice: when the NPU resumed from runtime suspend under memory pressure, command submission failed with `ENOMEM` and initially took the TTS service down. Keeping the device out of runtime suspend (a one-line udev rule) removed the failure.

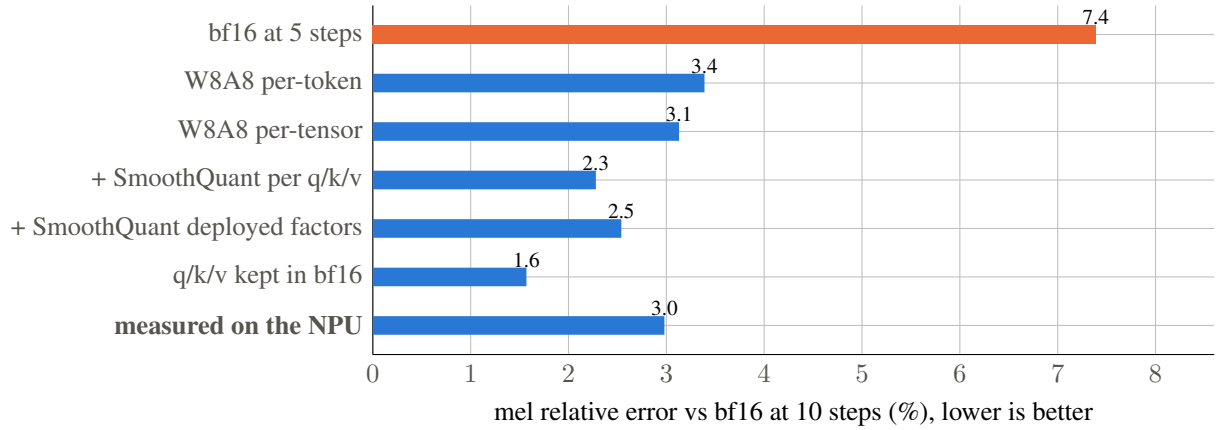


Figure 3: Mel deviation of each speed-up from the bf16 10-step reference (relative L_2; a numerical proxy, not a perceptual measure), one sentence, same seed and speech tokens. Orange: the CPU shortcut (half the flow steps, no quantization). Blue: int8 at the full 10 steps. All rows but the last simulate int8 GEMMs in PyTorch; the last is the deployed system on the NPU. "Per q/k/v" smooths each projection with its own factors, "deployed factors" uses the single shared q/k/v factor the NPU runs with; "q/k/v kept in bf16" leaves 62.5% of the GEMM work in int8.

3.3 Quantization

Weights are quantized symmetrically per output channel, activations per token (per row). To decide what to protect, we ran a fake-quantization study on the full pipeline: one 6.0 s Mandarin sentence, fixed seed and identical speech tokens in every arm, measuring the relative L_2 error of the final mel against CPU bf16 at 10 steps.

Figure 3 summarizes the result. Plain W8A8 deviates 3.1–3.4%, less than half the deviation from halving the flow steps (7.4%). Keeping one layer type at bf16 at a time shows where the error comes from: out-projection 3.66%, first feed-forward 2.88%, second feed-forward 3.11%, but q/k/v 1.57%. The q/k/v input is the AdaLN-modulated activation, where per-channel outliers concentrate. Instead of keeping q/k/v in bf16, which would move 37.5% of the GEMM work back to a slower path, we apply SmoothQuant [16] to that input only: each input channel j is divided by $s_j = \max|X_j|^{\alpha} / \max|W_j|^{1-\alpha}$ and the matching weight row multiplied by it, with $\alpha = 0.5$ (α of 0.3 and 0.7 gave 2.31% and 2.68%). The division folds into the host-side normalization step and costs nothing at run time. In simulation this gives 2.28%.

The deployed system measures 2.98%, and the 0.7-point gap has two parts we could separate. First, the smoothing factors: the NPU runs one fused q/k/v GEMM on a single quantized activation, so the export computes one shared factor per input channel from the concatenated q/k/v weights, calibrated on four held-out sentences (three Mandarin, one English). Simulating int8 with exactly those factors gives 2.54%; the shared formula alone, with the study's one-sentence calibration, gives 2.86%, so per-projection factors are worth about 0.6 points and the larger calibration set wins part of that back. Second, the remaining 0.44 points come from the runtime's own arithmetic, which the simulation leaves in PyTorch: attention packs queries, keys, values and softmax probabilities to bf16, softmax uses a polynomial exponential, and the AdaLN weights are stored in fp16. Per-projection smoothing on the NPU would require quantizing the activation three times per block; we leave it as future work.

3.4 Sharing the machine

The speech recognizer: residency and scheduling. We pair the TTS with whisper-xdna [11], which runs the encoder of Whisper large-v3-turbo [15] on the same NPU. Sharing the NPU raises two separate problems. *Residency*: both services must keep their overlays loaded, or every request pays reloading. With the 6-context ceiling, whisper-xdna is capped at 5 contexts (no measured speed loss: 2.57 s against 2.54–2.6 s) and the TTS holds 1. *Scheduling*: resident contexts still share the array, and with both active, recognition ran 30–40% slower. We protect recognition because it is on the user's critical path, the reply cannot start until the transcript exists, while synthesized audio has slack: a sentence only has to be ready when the previous one finishes playing, and the adaptive step count below absorbs delays. Since XDNA1 offers no preemption, synthesis yields cooperatively: a 16-byte shared-memory gate holds a holder count and an expiry time; the STT front end holds it around each transcription, and the DiT library checks it before each layer and waits while it is held. At that point every NPU run of the previous layer has completed and its results are in host buffers, so waiting changes no state and nothing is recomputed: holding the gate for 0.5 s paused a forward pass for 485–491 ms, and the output was bit-identical to an ungated pass. The expiry bounds the pause if a holder dies.

Recognition settings and memory. whisper-xdna decodes greedily without a prompt, which returned Chinese without punctuation and in mixed simplified and traditional script. We detect the language from the encoder output and, for Chinese only, give the decoder a one-sentence punctuated simplified-Chinese prompt, which it imitates; ASCII punctuation after a Chinese character is then made full-width. Beam search (width 5) corrected one short Mandarin clip that greedy decoding got wrong, but turned "cloned" into "clone" in an English one and added 0.2–0.8 s per request, so greedy remains the default. Memory mattered more than we expected on a 32 GB machine: whisper-xdna kept fp32 copies of every encoder matrix both as loaded and transposed, but reads only the transposed ones, and CosyVoice left several gigabytes of freed checkpoint copies in the allocator after loading. Dropping the unused copies and returning freed memory to the system took recognition from 8.0 to 5.9 GB and synthesis from 5.4 to 2.4 GB.

The LM backbone. CosyVoice3's LM is a Qwen2 transformer wrapped in CosyVoice's own logic: it builds the input from its own text and speech-token embedding tables and special tokens, predicts speech tokens with its own output head, and samples with its own rules. A text-generation API (token ids in, sampled text tokens out) would replace all of that. We cut at a narrower boundary and replace only the transformer:

```
cv_llm_reset()                          new utterance: clear the KV cache
h = cv_llm_step(E, n)   E: n × 896 input embeddings appended at the current position;
                        h: final-norm hidden state of the last position
```

CosyVoice calls the backbone exactly this way: the whole prompt once, then one token per step, and it applies its own head and sampler to h. The 62-line C shim implements these calls over llama.cpp [8]'s embedding-input batches with pooling disabled, and the exported GGUF drops the unused text output head. Q4_0 decodes 142 tokens/s on 4 threads and 122 on 2 when the machine is otherwise idle. Measured behaviour sets the thread counts: 4 threads instead of 2 cut per-sentence prefill from 1.2–1.5 s to 0.3–0.5 s, while decode at 2 threads costs the co-running 27B LLM less than at 4, so the shim switches between 4 prefill and 2 decode threads. Batching three sequences raised aggregate decode throughput only modestly (114 to 135 tokens/s idle, 58 to 60 next to the 27B), so requests stay serial. The GGUF export is byte-reproducible for a pinned llama.cpp commit.

Deadline-driven flow steps. In sentence-by-sentence playback, sentence $N+1$ only needs to be ready when sentence N stops playing. The server does not see the client's playback; it keeps an *estimated*

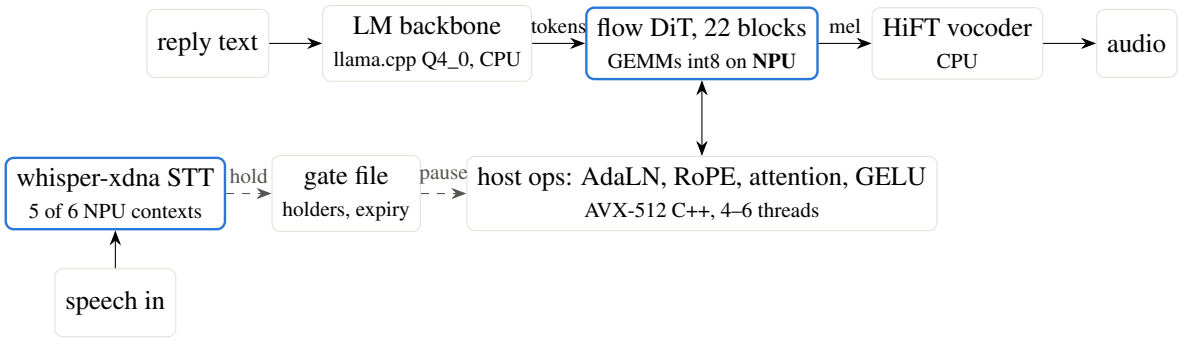


Figure 4: The TTS pipeline and its NPU neighbour. Blue outlines run on the NPU. The flow DiT uses one hardware context; whisper-xdna holds the gate while it transcribes, and the DiT waits at the next layer boundary.

playback end, assuming one listener who plays sentences in order, in real time, starting as soon as each is synthesized: after each sentence, $t_{\text{end}} \leftarrow \max(t_{\text{done}}, t_{\text{end}}) + T_{\text{audio}}$, where T_{audio} accounts for the requested speed (CosyVoice applies speed by resampling the mel after the flow). Before each flow call it predicts the cost as $c_0 \cdot T_{\text{audio}} + s \cdot F \cdot c_1$ for s steps over F frames, with c_1 updated online from observed calls, and picks the most steps in $[4, 10]$ that finish 0.3 s before t_{end} while still leaving a reserve of 2 s of synthesized audio queued for playback; steps above the minimum are bought only with time beyond that reserve, so a reply starts at the minimum and ramps up as the buffer grows. The first sentence of a reply, with nothing playing, gets a fixed 0.8 s budget. A request that arrives more than 6 s after the previous one starts a new reply: within a reply the client asks for the next sentence within a few seconds, while a new reply first waits for the user to finish speaking, for recognition and for the chat model's first sentence. Without this rule, a reply the user interrupted left its unplayed audio in the estimate, and the next reply's first sentence was given up to 10 steps as if that audio were still playing. If even 4 steps miss the deadline, it uses 4 and the listener hears a gap. The deviations in Figure 3 are for a fixed 10 steps; sentences synthesized with fewer steps, always including the first sentence of a reply, deviate more, and we did not measure the adaptive mode's quality. LM and flow also run as two pipelined stages with first-in-first-out ordering, so the LM of sentence $N{+}1$ overlaps the flow of sentence N. This only happens if the client keeps the next request in flight: Open WebUI's read-aloud requested one sentence at a time, so the stages never overlapped and each sentence cost LM plus flow back to back. Requesting one sentence ahead reduced the total playback gap of a simulated eight-sentence, 36 s read-aloud from 1.2–1.8 s to 0–0.8 s. The 2 s reserve did not measurably change it further (about 0.4 s on average with and without); the remaining gaps occur when a short first sentence is followed by a long one, before any buffer exists.

4 Evaluation

All measurements are on a Ryzen 7 8845HS (8 Zen 4 cores, XDNA1 NPU) 27 GiB of system memory shared with the integrated GPU, Linux 7.0 with the in-tree amdxdna driver, and XRT 2.21.75. "27B generating" means a 27B-parameter chat model decoding on the integrated GPU at the same time, as it would in a local voice chat. Real-time factor (RTF) is synthesis time divided by audio duration; below 1 is faster than real time.

Estimator. One full estimator call (22 blocks, CFG batch 2) over 318 frames takes 163 ms, and its output is within 3.3% of the fp32 PyTorch estimator. Over 478 frames the NPU call takes 200 ms against 546–558 ms for the PyTorch bf16 estimator on the CPU.

Table 4: Real-time factor of sentence synthesis, 10 flow steps (lower is better).

Setup	LLM idle	27B generating
CPU flow, serial, 3 sentences, Q8_0 LM, 4 threads	1.44	2.13
NPU flow, same setup	**0.84**	**1.33**
NPU flow, serial, 4 sentences, Q4_0 LM, 2 LM + 4 flow threads	0.85	1.32
same, LM and flow pipelined	**0.64**	**1.02**
CPU flow, 16 threads (whole CPU), 4 sentences	0.95–0.99	1.04–1.53

Table 5: Per-stage time of single sentences, deployed configuration, 10 flow steps (4 steps in parentheses).

Audio	LM	Flow (NPU)	Vocoder	Serial total
2.1 s	0.83 s	1.51 (0.61) s	0.21 s	2.56 (1.65) s
4.7 s	1.59 s	2.02 (0.81) s	0.44 s	4.05 (2.79) s
6.4 s	2.14 s	3.29 (1.31) s	0.62 s	6.05 (4.03) s
7.1 s	2.30 s	3.57 (1.42) s	0.68 s	6.56 (4.38) s

Sentence synthesis. Table 4 separates the effect of the NPU from that of the other changes. Under identical conditions (three sentences, serial, 4 threads, Q8_0 LM), moving the flow to the NPU lowers RTF from 1.44 to 0.84 with the machine otherwise idle, and from 2.13 to 1.33 while the 27B generates. The last two rows isolate pipelining in a second configuration (four sentences, Q4_0 LM, separate LM and flow threads): overlapping the stages lowers RTF from 0.85 to 0.64, and from 1.32 to 1.02. The two configurations differ in sentence count, LM quantization and threading, so their rows should not be compared with each other. The repository's single-sentence example runs at RTF 0.83 with the llama.cpp LM against 2.7 with CosyVoice's PyTorch LM.

Giving the CPU path every core does not close the gap (last row): even at 16 threads it is slower than the NPU path on 6, and it leaves nothing for the LLM. Serving speech continuously, the NPU configuration kept 4.4–4.5 cores busy on average against 6.7 for a 5-step CPU configuration.

Where the time goes. Table 5 times each stage of four sentences synthesized one at a time with the deployed configuration (speed 1.1, the 27B idle). The NPU call accounts for almost all of the flow; the PyTorch code around it (time and input embeddings, output projection) takes 0.1–0.2 s per sentence. At 10 steps the flow is about 55% of the serial time, the LM about 35% and the vocoder about 10%; with LM and flow pipelined, a 6.4 s sentence takes about 3.9 s. The reference voice's prompt (174 mel frames, 3.5 s) is a third of the flow's input for a sentence of this length.

Effect on the LLM. The 27B model decodes at 6.8 tokens/s alone. Running the TTS LM at 2 decode threads next to it costs it about 1.9 tokens/s (4.8–4.9 tokens/s remain); at 4 threads, 4.9; at 1 thread, 5.6; this, not the NPU, is the dominant interference, which is why decode uses 2 threads. The NPU GEMMs themselves slow by 24% while the 27B generates, against 45% for bf16, consistent with memory-bandwidth contention.

Coexistence with STT. Table 6 shows warm recognition latency for two clips with the gate in place. Recognition during active synthesis is within 1.5–5% (about 3% on average) of recognition alone; without the gate it was 30–40% slower.

Table 6: Warm STT latency (s), whisper-xdna large-v3-turbo with 5 NPU contexts, gate enabled.

Clip	STT alone	STT while TTS synthesizes
A	2.62, 2.57	2.66, 2.71
B	2.54, 2.64, 1.74	2.63, 2.67, 1.80

Voice loop. Behind an OpenAI-compatible chat front end with a 27B model, 16 spoken turns over 30 minutes completed without errors; STT took 2.52–2.62 s per turn and the TTS process used 1.8–2.5 GB. With the 27B idle, adaptive steps brought the first audio of a reply from 4.1–4.8 s (fixed 10 steps) to 3.0–3.8 s, at the cost of a 4-step first sentence.

Complete application. Packaged with the 27B (llama.cpp, Vulkan, 32K context) and Open WebUI on a 32 GB machine, all services together use about 25 GB of the 27.7 GB Linux sees. In a scripted voice call (synthetic speech as the microphone, three turns, run twice), the reply started playing 8.3–11.8 s after the end of the question in a running conversation, and 20.5 s for the very first message after the services start, when the 27B first reads Open WebUI's 700-token setup prompt. Of a later turn, recognition takes about 3.3 s, the chat model's first sentence 2–3 s and the synthesis of that sentence 4–5 s, since the chat model is still generating; the call waits for 1.2 s of silence before it considers the question finished, and starts recognition speculatively after 0.85 s. Under that load most sentences get 4–6 flow steps.

Reproducibility. Weight export, GGUF export and overlay compilation are deterministic: rebuilding from scratch in the published container produced int8 weights, a GGUF and an NPU instruction stream bit-identical to the deployed ones. The repository includes a bit-exact GEMM self-test and an estimator check against PyTorch.

5 Limitations

- **One machine, one generation.** All numbers come from one Ryzen 7 8845HS. The overlay compiles for XDNA2 (Ryzen AI 300; 8 columns), but we have no XDNA2 hardware to run it.
- **Objective quality only.** Mel deviation is a numerical proxy, not a perceptual loss. Listening was informal; we have no MOS or WER study. The fake-quantization study used one sentence, and the quality of adaptive-step synthesis was not measured.
- **Two RTF configurations.** Table 4 isolates the NPU (first configuration) and pipelining (second configuration) separately; the combined effect of all changes is not a single controlled comparison.
- **Streaming.** The library supports CosyVoice's block-causal streaming mask, but streamed output differed from whole-sentence output by 24% at the estimator and sounded worse, so the deployment synthesizes sentence by sentence.
- **Power.** We did not measure NPU power or energy per utterance.
- **Context ceiling.** Six hardware contexts per device limit how many NPU services can coexist; ours uses all six.

6 Related work

CosyVoice [5–7] combines an LLM over supervised semantic speech tokens with a flow-matching decoder in the style of Matcha-TTS [13]. Post-training int8 quantization of transformers is well studied for LLMs [4, 16]; we apply SmoothQuant to a diffusion transformer and find a single input carries most of the error. whisper-xdna [11] showed that the XDNA1 NPU can run a full Whisper encoder from Linux through MLIR-AIE [2] overlays; we build on its container and serve alongside it.

7 Availability

Code, the C++ DiT runtime, the llama.cpp shim, export and test scripts are at `https://github.com/YuhuaBillChen/npu-cosyvoice` under Apache-2.0. Demo videos and audio samples, including a voice cloned from a 7.5 s public-domain LJ Speech [10] recording, are at `https://yuhuabillchen.github.io/npu-cosyvoice/`.

Running it. On a Ryzen AI 7040/8040 machine with Linux 6.14+ and Docker, a docker-compose bundle prepares the model, weights and NPU kernel once and then serves `/v1/audio/speech` and `/v1/audio/transcriptions`, so any client of the OpenAI audio API can use it. Custom voices need a 5–15 s recording and its transcript. The repository's GEMM self-test (bit-exact) and estimator check (against PyTorch) are the first things to run on new hardware.

Where it applies. Tested on one XDNA1 machine. XDNA2 builds but is untested. The NPU path serves one synthesis at a time and occupies one of six NPU contexts; it pays off when the CPU is needed by something else, such as a local LLM.

Acknowledgments

AI assistants (Anthropic Claude and OpenAI Codex) were used for parts of the implementation, for running and analysing experiments, and for drafting this report. The author designed the system, reviewed the code, results and text, and is responsible for the content.

References

[1] AMD. amdxdna: Linux kernel driver for AMD XDNA NPUs. Linux kernel, `drivers/accel/amdxdna`, mainline since 6.14, 2025.

[2] AMD Research and Advanced Development. MLIR-AIE: An MLIR-based toolchain for AMD AI engine-enabled devices. `https://github.com/Xilinx/mlir-aie`, 2026.

[3] Yuhua Chen. JustFit: Just-in-time state management for local LLM serving. arXiv:2609.17475, 2026.

[4] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL `https://arxiv.org/abs/2208.07339`.

[5] Zhihao Du et al. CosyVoice: A scalable multilingual zero-shot text-to-speech synthesizer based on supervised semantic tokens. *arXiv preprint arXiv:2407.05407*, 2024. URL `https://arxiv.org/abs/2407.05407`.

[6] Zhihao Du et al. CosyVoice 2: Scalable streaming speech synthesis with large language models. *arXiv preprint arXiv:2412.10117*, 2024. URL https://arxiv.org/abs/2412.10117.

[7] Zhihao Du et al. CosyVoice 3: Towards in-the-wild speech generation via scaling-up and post-training. *arXiv preprint arXiv:2505.17589*, 2025. URL https://arxiv.org/abs/2505.17589.

[8] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp, 2026.

[9] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. URL https://arxiv.org/abs/2207.12598.

[10] Keith Ito and Linda Johnson. The LJ speech dataset. https://keithito.com/LJ-Speech-Dataset/, 2017.

[11] Mikhail Kostryukov. whisper-xdna: Whisper speech recognition on the AMD XDNA NPU. https://github.com/drakosha/whisper-xdna, 2026.

[12] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.02747.

[13] Shivam Mehta, Ruibo Tu, Jonas Beskow, Éva Székely, and Gustav Eje Henter. Matcha-TTS: A fast TTS architecture with conditional flow matching. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024. URL https://shivammehta25.github.io/Matcha-TTS/.

[14] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. URL https://openaccess.thecvf.com/content/ICCV2023/html/Peebles_Scalable_Diffusion_Models_with_Transformers_ICCV_2023_paper.html.

[15] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning (ICML)*, 2023. URL https://proceedings.mlr.press/v202/radford23a.html.

[16] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning (ICML)*, 2023. URL https://proceedings.mlr.press/v202/xiao23c.html.